ShieldLabs Security Report

https://github.com/udip15/test-repoo-shieldlabs.git

76 Good		
	Scan Type	CODE
	Scan ID	scan_4b0b71cc
	Generated	2026-07-01 09:37 UTC
	Total Findings	4
	Attack Chains	0

Severity Breakdown

Critical & High-Risk Findings - Immediate Attention Required

HIGH

SQL Injection

C:\Users\acer\AppData\Local\Temp\shieldlabs_repo_m30yei8h\app.py:6

HIGH

Hardcoded Secret

C:\Users\acer\AppData\Local\Temp\shieldlabs_repo_m30yei8h\app.py:12

HIGH

Command Injection

C:\Users\acer\AppData\Local\Temp\shieldlabs_repo_m30yei8h\app.py:10

HIGH

Weak Hashing

C:\Users\acer\AppData\Local\Temp\shieldlabs_repo_m30yei8h\auth.py:5

Findings Summary

Severity	Vulnerability	Location	CVSS	
HIGH	SQL Injection	C:\Users\acer\AppData\Local\Ten	9.3	pp.py:6
HIGH	Hardcoded Secret	C:\Users\acer\AppData\Local\Ten	8.1	pp.py:12
HIGH	Command Injection	C:\Users\acer\AppData\Local\Ten	10.0	pp.py:10
HIGH	Weak Hashing	C:\Users\acer\AppData\Local\Ten	5.8	uth.py:5

Detailed Findings

1. SQL Injection

File: C:\Users\acer\AppData\Local\Temp\shieldlabs_repo_m30yei8h\app.py:6 | CVSS: 9.3 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:L)

Vulnerable code:

```
query = "SELECT * FROM users WHERE name = '" + username + "'"
```

Suggested fix:

```
query = "SELECT * FROM users WHERE name = %s"
```

AI Review: The proposed fix is a good start but needs additional safeguards to handle edge cases and potential malicious inputs. Ensure that `username` is validated and sanitized before constructing the SQL query.

Business Impact: Complete data loss or theft if not detected and mitigated promptly.

2. Hardcoded Secret

File: C:\Users\acer\AppData\Local\Temp\shieldlabs_repo_m30yei8h\app.py:12 | CVSS: 8.1 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N)

Vulnerable code:

```
API_KEY = "sk-live-51H8xJ2eZvKYlo2C0FAKEKEYFORTESTING"
```

Suggested fix:

```
API_KEY = os.getenv('SHIELDLABS_API_KEY')
```

AI Review: The proposed fix is safe and complete. By using `os.getenv('SHIELDLABS_API_KEY')`, the code retrieves the API key from an environment variable instead of hardcoding it directly, which mitigates the risk of exposing sensitive information in the source code.

Business Impact: A hardcoded credential or key in a startup web application can lead to unauthorized access to critical systems and data, potentially causing financial losses, reputational damage, and legal issues.

3. Command Injection

File: C:\Users\acer\AppData\Local\Temp\shieldlabs_repo_m30yei8h\app.py:10 | CVSS: 10.0 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H)

Vulnerable code:

```
os.system("ping " + user_input)
```

Suggested fix:

```
os.system(f'ping {shlex.quote(user_input)}')
```

AI Review: The proposed fix using `shlex.quote(user_input)` within an f-string is safe and complete. This approach ensures that any special characters in `user_input` are properly escaped, preventing command injection attacks. There are no remaining concerns with this fix.

Business Impact: Complete system compromise, potential loss of sensitive data and intellectual property

4. Weak Hashing

File: C:\Users\acer\AppData\Local\Temp\shieldlabs_repo_m30yei8h\auth.py:5 | CVSS: 5.8 (CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N)

Vulnerable code:

```
return hashlib.md5(password.encode()).hexdigest()
```

Suggested fix:

```
return hashlib.sha256(password.encode()).hexdigest()
```

AI Review: The proposed fix is safe and complete. It replaces the vulnerable MD5 hashing algorithm with the much stronger SHA-256, enhancing the security of password storage. No additional concerns are noted.

Business Impact: The attacker could potentially gain unauthorized access to user accounts, leading to data breaches and potential financial loss.